

React — Complete Concept Guide

In-Depth Theory + Multiple Examples, Company-Relevant Focus

Prepared for K N | Building on Your JavaScript Foundation

Table of Contents

- 1. Fundamentals
- 2. Hooks
- 3. Component Patterns
- 4. State Management
- 5. Forms
- 6. Side Effects & Data Fetching
- 7. Performance Optimization
- 8. Routing
- 9. Advanced Patterns
- 10. Ecosystem Essentials

1. Fundamentals

1.1 JSX Syntax & Rules

JSX (JavaScript XML) lets you write HTML-like markup directly inside JavaScript. Under the hood, JSX compiles to `React.createElement()` calls — it's not a template language, it's actual JavaScript with a friendlier syntax. Key rules: one root element per return (or a Fragment), use `className` instead of `class`, and any JS expression can go inside curly braces `{ }`.

Example 1 — basic JSX

```
function Greeting() {
  const name = "Karan";
  return <h1>Hello, {name}!</h1>;
}
// compiles to: React.createElement('h1', null, 'Hello, ', name, '!')
```

Example 2 — embedding expressions

```
function Price() {
  const price = 499;
  const discount = 50;
  return (
    <p>Final price: ₹{price - discount}</p>
  );
}
// any valid JS expression works inside {} - math, function calls, ternaries
```

Example 3 — multiple elements need a wrapper (Fragment)

```
// WRONG - two sibling elements without a parent
// return

Title

Text

; -> Syntax Error

// RIGHT - wrap in a Fragment (doesn't add extra DOM nodes)
function Card() {
  return (
    <>
      <h1>Title</h1>
      <p>Text</p>
    </>
  );
}
```

Example 4 — className and self-closing tags

```
function Avatar() {
  return (
    
  );
}
// className not class; self-closing tags need the trailing slash
}
```

Gotcha: JSX requires every tag to be closed, even ones that are unclosed in plain HTML like `<img>` or `<br>` — they must be written as `<img />` and `<br />`.

1.2 Components (Function Components)

A component is a reusable, independent piece of UI — essentially a JavaScript function that returns JSX. Modern React almost exclusively uses **function components** (older codebases may still use class components, but hooks made those largely unnecessary). Component names MUST start with a capital letter — React uses this to distinguish components from regular HTML tags.

Example 1 — a basic function component

```
function Welcome() {
  return <h1>Welcome to the app!</h1>;
}
// used elsewhere as: <Welcome /> (note capital W)
```

Example 2 — composing components together

```
function Header() {
  return <header><h1>My App</h1></header>;
}
function Footer() {
  return <footer>© 2026</footer>;
}
function App() {
  return (
    <>
      <Header />
      <p>Main content here</p>
      <Footer />
    </>
  );
}
```

Example 3 — why lowercase vs capital matters

```
// lowercase tag -> React treats it as a built-in HTML element
<div></div>

// Capitalized tag -> React looks for a component named Div
<Div></Div>
// if you write a component "profile" lowercase, React will try
// to render it as an HTML tag instead of your component
```

1.3 Props

Props (short for "properties") are how data flows INTO a component from its parent — similar to function arguments. Props are read-only; a component must never modify its own props directly. This one-directional data flow (parent → child) is core to how React apps are structured.

Example 1 — passing and receiving props

```
function UserCard(props) {
  return <p>{props.name} is {props.age} years old</p>;
}

// usage:
<UserCard name="Karan" age={21} />
```

Example 2 — destructuring props (the common style)

```
function UserCard({ name, age }) {
  return <p>{name} is {age} years old</p>;
}
// cleaner - directly extracts the fields you need
```

Example 3 — default prop values

```
function Button({ label = "Click me", color = "blue" }) {
  return <button style={{ color }}>{label}</button>;
}
<Button /> // renders "Click me" in blue - uses defaults
<Button label="Submit" /> // overrides label only
```

Example 4 — passing functions as props (parent-child communication)

```
function Child({ onClick }) {
  return <button onClick={onClick}>Notify Parent</button>;
}
function Parent() {
  function handleClick() { console.log("Child clicked!"); }
  return <Child onClick={handleClick} />;
}
// this is how children notify parents of events - passing callbacks down
```

Gotcha: Never do `props.name = "New"` inside a component — props are immutable from the child's perspective. If data needs to change, it should live in state (in the parent) and be updated via a passed-down function.

1.4 Rendering & the Virtual DOM

React doesn't touch the real DOM directly on every update — it maintains a lightweight in-memory copy called the **Virtual DOM**. When state/props change, React creates a new virtual tree, compares ("diffs") it against the previous one, and only updates the real DOM nodes that actually changed. This process is called **reconciliation**, and it's why React is fast even with frequent updates.

Example 1 — what triggers a re-render

```
function Counter() {
  const [count, setCount] = useState(0);
  return (
    <button onClick={() => setCount(count + 1)}>
      Count: {count}
    </button>
  );
}
// calling setCount triggers React to re-run this function and
// compare the new JSX output against the previous render
```

Example 2 — only changed DOM nodes update

```
function App() {
  const [count, setCount] = useState(0);
  return (
    <div>
      <h1>Static Title</h1> {/* never re-rendered to real DOM, unchanged */}
      <p>Count: {count}</p> {/* only this text node updates in real DOM */}
      <button onClick={() => setCount(c => c+1)}>+</button>
    </div>
  );
}
```

Why it matters: Understanding that React re-runs your ENTIRE component function on every state change (not just a piece of it) explains why you need `useMemo/useCallback` for expensive operations, and why closures inside components capture that render's specific values.

1.5 Conditional Rendering

Since JSX is just JavaScript, you use normal JS constructs — `if`, ternary operators, and `&&` — to decide what to render based on a condition.

Example 1 — if/else via early return

```
function Greeting({ isLoggedIn }) {
  if (isLoggedIn) {
    return <h1>Welcome back!</h1>;
  }
  return <h1>Please log in</h1>;
}
```

Example 2 — ternary inside JSX

```
function StatusBadge({ isOnline }) {
  return (
    <span>{isOnline ? "🟢 Online" : "🔴 Offline"}</span>
  );
}
```

Example 3 — && for "render only if true"

```
function Inbox({ unreadCount }) {
  return (
    <div>
      <h2>Inbox</h2>
      {unreadCount > 0 && <span>You have {unreadCount} unread</span>}
    </div>
  );
}
```

Gotcha: `{count && <Badge />}` will render a literal "0" if count is 0, since 0 is falsy but still gets rendered by React as text. Use `{count > 0 && ...}` instead.

1.6 Lists & Keys

To render a list of items, you typically use `.map()` to transform an array of data into an array of JSX elements. Each item needs a unique `key` prop so React can track which items changed, were added, or removed — this is essential for correct and efficient re-rendering.

Example 1 — rendering a list with map

```
function TodoList({ todos }) {
  return (
    <ul>
      {todos.map(todo => (
        <li key={todo.id}>{todo.text}</li>
      ))}
    </ul>
  );
}
```

Example 2 — why array index as key is risky

```
// RISKY - using index as key
{todos.map((todo, index) => <li key={index}>{todo.text}</li>)}
// if items are reordered/removed, React may mismatch state to the wrong item

// SAFE - using a stable, unique id from your data
{todos.map(todo => <li key={todo.id}>{todo.text}</li>)}
```

Example 3 — keys with more complex list items

```
function ProductGrid({ products }) {
  return (
    <div className="grid">
      {products.map(p => (
        <div key={p.id} className="card">
          <h3>{p.name}</h3>
          <p>₹{p.price}</p>
        </div>
      ))}
    </div>
  );
}
// key goes on the OUTERMOST element returned inside map, not deeper children
```

2. Hooks

2.1 useState

`useState` lets a function component hold and update its own local data ("state") between renders. Calling the setter function triggers React to re-render the component with the new value. State updates are ASYNCHRONOUS and BATCHED — React may group multiple `setState` calls together before re-rendering, for performance.

Example 1 — basic counter

```
import { useState } from "react";

function Counter() {
  const [count, setCount] = useState(0); // [currentValue, setterFunction]

  return (
    <div>
      <p>Count: {count}</p>
      <button onClick={() => setCount(count + 1)}></button>
    </div>
  );
}
```

Example 2 — functional updates (important when new state depends on old)

```
function Counter() {
  const [count, setCount] = useState(0);

  function incrementTwice() {
    // WRONG - both calls use the SAME stale 'count' value
    setCount(count + 1);
    setCount(count + 1);
    // result: count only goes up by 1, not 2
  }

  function incrementTwiceCorrect() {
    // RIGHT - functional form always gets the LATEST value
    setCount(prev => prev + 1);
    setCount(prev => prev + 1);
    // result: count correctly goes up by 2
  }
}
```

Example 3 — state with objects and arrays (immutable updates)

```
function ProfileForm() {
  const [user, setUser] = useState({ name: "", age: 0 });

  function updateName(newName) {
    // must spread - never mutate state directly
    setUser(prev => ({ ...prev, name: newName }));
  }

  const [items, setItems] = useState([]);
  function addItem(item) {
    setItems(prev => [...prev, item]); // new array, not push()
  }
}
```

Example 4 — lazy initial state (for expensive initial values)

```
// WRONG - runs the expensive function on EVERY render
const [data, setData] = useState(computeExpensiveValue());

// RIGHT - pass a function, React only calls it ONCE, on mount
const [data, setData] = useState(() => computeExpensiveValue());
```

Gotcha: Calling `setCount(count + 1)` doesn't update `count` immediately within the same function — `count` still holds the OLD value until the next render. Logging `count` right after calling the setter will show the stale value.

2.2 useEffect

`useEffect` lets you run side effects — code that interacts with something outside of React's rendering, like data fetching, subscriptions, timers, or manually changing the DOM. It runs AFTER the component renders. The dependency array (second argument) controls WHEN it re-runs.

Example 1 — the three dependency array patterns

```
import { useEffect } from "react";

useEffect(() => {
  console.log("Runs after EVERY render");
}); // no dependency array

useEffect(() => {
  console.log("Runs only ONCE, after first mount");
}, []); // empty dependency array

useEffect(() => {
  console.log("Runs on mount AND whenever 'userId' changes");
}, [userId]); // dependency array with values
```

Example 2 — data fetching on mount

```
function UserProfile({ userId }) {
  const [user, setUser] = useState(null);

  useEffect(() => {
    async function loadUser() {
      const res = await fetch(`/api/users/${userId}`);
      const data = await res.json();
      setUser(data);
    }
    loadUser();
  }, [userId]); // re-fetches whenever userId changes

  return user ? <p>{user.name}</p> : <p>Loading...</p>;
}
```

Example 3 — cleanup functions (preventing memory leaks)

```
function Timer() {
  const [seconds, setSeconds] = useState(0);

  useEffect(() => {
    const interval = setInterval(() => {
      setSeconds(prev => prev + 1);
    }, 1000);

    return () => {
      clearInterval(interval); // cleanup - runs before next effect OR on unmount
    };
  }, []);

  return <p>Seconds: {seconds}</p>;
}
// without cleanup, the interval keeps running even after component unmounts
```

Example 4 — avoiding infinite loops with `useEffect`

```
// WRONG - creates an infinite loop
function BadExample() {
  const [count, setCount] = useState(0);
  useEffect(() => {
    setCount(count + 1); // triggers re-render -> effect runs again -> forever
  }); // no dependency array = runs after every render
}

// RIGHT - only runs once, or add a proper condition/dependency
useEffect(() => {
  setCount(1);
}, []);
```

Gotcha: Forgetting to add a variable used inside `useEffect` to its dependency array is one of the most common React bugs — it causes the effect to use a stale ("frozen") value from an old render. ESLint's [react-hooks/exhaustive-deps](#) rule catches this.

2.3 `useRef`

`useRef` creates a mutable object that persists across renders WITHOUT causing a re-render when it changes. Two main uses: (1) directly accessing a DOM element (like focusing an input), and (2) storing a mutable value that shouldn't trigger a re-render (like a previous value or a timer ID).

Example 1 — accessing a DOM element

```
import { useRef } from "react";

function FocusInput() {
  const inputRef = useRef(null);

  function handleClick() {
    inputRef.current.focus(); // directly calling a DOM method
  }

  return (
    <>
      <input ref={inputRef} type="text" />
      <button onClick={handleClick}>Focus Input</button>
    </>
  );
}
```

Example 2 — storing a value without triggering re-render

```
function RenderCounter() {
  const renderCount = useRef(0);

  useEffect(() => {
    renderCount.current += 1;
    console.log(`Rendered ${renderCount.current} times`);
  });
  // updating renderCount.current does NOT cause a re-render,
  // unlike useState which always does
}
```

Example 3 — tracking previous value

```
function PreviousValueTracker({ value }) {
  const prevValue = useRef();

  useEffect(() => {
    prevValue.current = value; // stores current AFTER render, becomes "previous" next time
  });

  return <p>Now: {value}, Before: {prevValue.current}</p>;
}
```

Why `useRef` vs `useState`: Use `useState` when the value affects what's rendered on screen. Use `useRef` when you need to persist a value across renders but changing it should NOT cause a re-render (like a timer ID, previous value, or DOM reference).

2.4 `useContext`

`useContext` lets a component read a value from a Context Provider higher up in the tree, without needing it passed down manually through every intermediate component (avoiding "prop drilling"). You first create context with `createContext`, wrap your tree with a `Provider`, then any descendant can subscribe with `useContext`.

Example 1 — creating and providing context

```
import { createContext, useContext, useState } from "react";

const ThemeContext = createContext();

function App() {
  const [theme, setTheme] = useState("light");
  return (
    <ThemeContext.Provider value={{ theme, setTheme }}>
      <Toolbar />
    </ThemeContext.Provider>
  );
}
```

Example 2 — consuming context deep in the tree

```
function Toolbar() {
```

```

    return <ThemeButton />; // no need to pass theme through here
  }

function ThemeButton() {
  const { theme, setTheme } = useContext(ThemeContext);
  return (
    <button onClick={() => setTheme(theme === "light" ? "dark" : "light")}>
      Current theme: {theme}
    </button>
  );
}
// ThemeButton reads context directly, skipping Toolbar entirely

```

Example 3 — a custom hook wrapper for cleaner usage

```

function useTheme() {
  const context = useContext(ThemeContext);
  if (!context) throw new Error("useTheme must be used within ThemeProvider");
  return context;
}
// usage elsewhere: const { theme, setTheme } = useTheme();

```

Gotcha: Every component that uses [useContext](#) re-renders whenever the context value changes — even if that specific component only cares about part of it. For frequently-changing values, consider splitting contexts or using a dedicated state library.

2.5 useReducer

[useReducer](#) is an alternative to [useState](#) for managing more complex state logic — especially when the next state depends on the previous one in non-trivial ways, or when multiple related pieces of state update together. It follows the same reducer pattern as Redux: you dispatch an "action," and a pure reducer function decides the new state.

Example 1 — basic reducer setup

```

import { useReducer } from "react";

function reducer(state, action) {
  switch (action.type) {
    case "increment": return { count: state.count + 1 };
    case "decrement": return { count: state.count - 1 };
    case "reset": return { count: 0 };
    default: throw new Error("Unknown action");
  }
}

function Counter() {
  const [state, dispatch] = useReducer(reducer, { count: 0 });
  return (
    <div>
      <p>{state.count}</p>
      <button onClick={() => dispatch({ type: "increment" })}>+</button>
      <button onClick={() => dispatch({ type: "decrement" })}>-</button>
    </div>
  );
}

```

Example 2 — managing a form with multiple fields

```

function formReducer(state, action) {
  switch (action.type) {
    case "UPDATE_FIELD":
      return { ...state, [action.field]: action.value };
    case "RESET":
      return { name: "", email: "" };
    default: return state;
  }
}

function SignupForm() {
  const [state, dispatch] = useReducer(formReducer, { name: "", email: "" });

  return (
    <input
      value={state.name}
      onChange={(e) => dispatch({ type: "UPDATE_FIELD", field: "name", value: e.target.value })}
    />
  );
}

```

useState vs useReducer: Use [useState](#) for simple, independent values. Switch to [useReducer](#) when you have multiple sub-values that update together, or complex update logic that's easier to centralize and test as a pure function.

2.6 useMemo

[useMemo](#) caches (memoizes) the RESULT of an expensive calculation, and only recomputes it when one of its listed dependencies changes. On every other re-render, it returns the cached value instead of recalculating.

Example 1 — avoiding expensive recalculation

```

import { useMemo } from "react";

function ProductList({ products, searchTerm }) {
  const filtered = useMemo(() => {
    console.log("Filtering..."); // only logs when products or searchTerm change
    return products.filter(p => p.name.includes(searchTerm));
  }, [products, searchTerm]);

  return filtered.map(p => <p key={p.id}>{p.name}</p>);
}

```

Example 2 — without useMemo (the problem it solves)

```

function Dashboard({ items, theme }) {
  // WITHOUT useMemo: this recalculates on EVERY render,
  // even when only 'theme' changes and 'items' didn't
  const total = items.reduce((sum, i) => sum + i.price, 0);

  // WITH useMemo: only recalculates when 'items' actually changes
  const totalMemoized = useMemo(() => {
    return items.reduce((sum, i) => sum + i.price, 0);
  }, [items]);
}

```


Gotcha: Don't wrap every calculation in `useMemo` — it has its own overhead (memory for cache, comparison cost). Reserve it for genuinely expensive computations or to preserve reference equality for dependencies of other hooks.

2.7 useCallback

`useCallback` is like `useMemo`, but specifically for memoizing FUNCTIONS instead of values. Without it, a new function is created on every render — which can cause child components wrapped in `React.memo` to re-render unnecessarily, since they see a "new" function prop each time.

Example 1 — the problem `useCallback` solves

```
function Parent() {
  const [count, setCount] = useState(0);

  // WITHOUT useCallback: a brand new function every render
  const handleClick = () => console.log("clicked");

  return <ExpensiveChild onClick={handleClick} />;
  // ExpensiveChild re-renders every time, even if wrapped in React.memo,
  // because handleClick is a "different" function reference each render
}
```

Example 2 — fixing it with `useCallback`

```
import { useCallback } from "react";

function Parent() {
  const [count, setCount] = useState(0);

  const handleClick = useCallback(() => {
    console.log("clicked");
  }, []); // same function reference across renders (empty deps)

  return <ExpensiveChild onClick={handleClick} />;
  // now ExpensiveChild (if wrapped in React.memo) won't re-render needlessly
}
```

Example 3 — `useCallback` with dependencies

```
function SearchBox({ onSearch }) {
  const [query, setQuery] = useState("");

  const handleSearch = useCallback(() => {
    onSearch(query);
  }, [query, onSearch]); // recreated only when query or onSearch changes
}
```

`useMemo` vs `useCallback`: `useMemo(fn, deps)` returns the VALUE that `fn()` returns. `useCallback(fn, deps)` returns the FUNCTION `fn` itself, unchanged. In fact, `useCallback(fn, deps)` is equivalent to `useMemo(() => fn, deps)`.

2.8 Custom Hooks

A custom hook is simply a JavaScript function whose name starts with `use` and that calls other hooks inside it. Custom hooks let you extract and reuse stateful logic across multiple components — without repeating code or wrapping components (unlike older patterns like HOCs).

Example 1 — a custom hook for fetching data

```
function useFetch(url) {
  const [data, setData] = useState(null);
  const [loading, setLoading] = useState(true);
  const [error, setError] = useState(null);

  useEffect(() => {
    setLoading(true);
    fetch(url)
      .then(res => res.json())
      .then(data => { setData(data); setLoading(false); })
      .catch(err => { setError(err); setLoading(false); });
  }, [url]);

  return { data, loading, error };
}
```

Example 2 — using the custom hook (clean, reusable)

```
function UserProfile({ userId }) {
  const { data: user, loading, error } = useFetch(`/api/users/${userId}`);

  if (loading) return <p>Loading...</p>;
  if (error) return <p>Error loading user</p>;
  return <p>{user.name}</p>;
}

// same hook reused elsewhere:
function PostList() {
  const { data: posts, loading } = useFetch("/api/posts");
}
```

Example 3 — custom hook for window size (common utility pattern)

```
function useWindowWidth() {
  const [width, setWidth] = useState(window.innerWidth);

  useEffect(() => {
    function handleResize() { setWidth(window.innerWidth); }
    window.addEventListener("resize", handleResize);
    return () => window.removeEventListener("resize", handleResize);
  }, []);

  return width;
}

// usage: const width = useWindowWidth();
```

Why it matters: Custom hooks are the standard way companies share logic across components — data fetching, form handling, auth checks, debouncing, etc. Interviewers frequently ask you to build one from scratch.

3. Component Patterns

3.1 Composition vs Inheritance

React strongly favors **composition** over inheritance (unlike traditional OOP UI frameworks). Instead of building complex components by extending a base class, you build them by combining smaller, focused components together — passing components as props or children.

Example 1 — composing instead of inheriting

```
// Not the React way: class Dialog extends BaseDialog { ... }

// The React way: compose smaller pieces
function Dialog({ title, children }) {
  return (
    <div className="dialog">
      <h2>{title}</h2>
      {children}
    </div>
  );
}

function WelcomeDialog() {
  return (
    <Dialog title="Welcome">
      <p>Thanks for signing up!</p>
    </Dialog>
  );
}
```

Example 2 — specialization through composition

```
function Button({ children, ...props }) {
  return <button className="btn" {...props}>{children}</button>;
}

function DangerButton({ children, ...props }) {
  return <Button className="btn-danger" {...props}>{children}</Button>;
}

// DangerButton composes Button rather than extending a class
```

3.2 The children Prop

children is a special prop that automatically holds whatever is nested between a component's opening and closing tags. It enables flexible, reusable "wrapper" or "container" components (like Cards, Modals, Layouts) that don't need to know what's inside them.

Example 1 — basic children usage

```
function Card({ children }) {
  return <div className="card">{children}</div>;
}

// usage - anything between the tags becomes 'children'
<Card>
  <h2>Title</h2>
  <p>Some description text</p>
</Card>
```

Example 2 — a reusable Modal using children

```
function Modal({ isOpen, onClose, children }) {
  if (!isOpen) return null;
  return (
    <div className="modal-overlay">
      <div className="modal">
        <button onClick={onClose}>×</button>
        {children}
      </div>
    </div>
  );
}

// reused with completely different content each time:
<Modal isOpen={showModal} onClose={() => setShowModal(false)}>
  <LoginForm />
</Modal>
```

3.3 Controlled vs Uncontrolled Components

A **controlled component** has its value fully managed by React state — every change goes through **onChange** and updates state, which then reflects back to the input's value. An **uncontrolled component** manages its own internal state in the DOM, and you read its value only when needed (usually via a ref). Companies mostly use controlled components for forms because they're more predictable.

Example 1 — controlled input

```
function ControlledInput() {
  const [value, setValue] = useState("");

  return (
    <input
      value={value}
      onChange={(e) => setValue(e.target.value)}
    />
  );
}

// React state is the single source of truth for the input's value
```

Example 2 — uncontrolled input with ref

```
function UncontrolledInput() {
  const inputRef = useRef(null);

  function handleSubmit() {
    console.log(inputRef.current.value); // read directly from the DOM
  }

  return (
    <>
      <input ref={inputRef} defaultValue="Initial text" />
      <button onClick={handleSubmit}>Submit</button>
    </>
  );
}
```

```
});  
}
```

When to use which: Controlled components are preferred for validation, conditional disabling, and syncing with other UI. Uncontrolled components are simpler for basic forms where you just need the value on submit, or for integrating with non-React libraries.

3.4 Lifting State Up

When two or more sibling components need to share and stay in sync with the same data, you move ("lift") that state to their closest common parent, then pass it down via props. This ensures a single source of truth instead of duplicated, out-of-sync state.

Example 1 — before lifting (broken, out of sync)

```
// each has its OWN separate state - they can't stay in sync  
function TemperatureInputCelsius() {  
  const [temp, setTemp] = useState("");  
  // ...  
}  
function TemperatureInputFahrenheit() {  
  const [temp, setTemp] = useState("");  
  // ...  
}
```

Example 2 — after lifting state to the parent

```
function TemperatureConverter() {  
  const [celsius, setCelsius] = useState("");  
  
  const fahrenheit = celsius ? (celsius * 9/5) + 32 : "";  
  
  return (  
    <>  
      <input value={celsius} onChange={e => setCelsius(e.target.value)} />  
      <p>Fahrenheit: {fahrenheit}</p>  
    </>  
  );  
}  
// single source of truth (celsius) - fahrenheit is always derived, always in sync
```

3.5 Prop Drilling

Prop drilling happens when you pass data through many layers of components that don't actually need it themselves — just to reach a deeply nested child that does. It makes code harder to maintain. Context API (or a state management library) solves this by letting deeply nested components read data directly.

Example 1 — the problem: passing through unrelated components

```
function App() {  
  const [user, setUser] = useState({ name: "Karan" });  
  return <Layout user={user} />;  
}  
function Layout({ user }) {  
  return <Sidebar user={user} />; // Layout doesn't use 'user' itself  
}  
function Sidebar({ user }) {  
  return <UserAvatar user={user} />; // Sidebar doesn't use it either  
}  
function UserAvatar({ user }) {  
  return <img src={user.avatarUrl} />; // only THIS one actually needs 'user'  
}
```

Example 2 — solved with Context (no more drilling)

```
const UserContext = createContext();  
  
function App() {  
  const [user, setUser] = useState({ name: "Karan" });  
  return (  
    <UserContext.Provider value={user}>  
      <Layout />    {/* no prop needed here anymore */}  
    </UserContext.Provider>  
  );  
}  
function UserAvatar() {  
  const user = useContext(UserContext); // reads directly, skipping Layout & Sidebar  
  return <img src={user.avatarUrl} />;  
}
```

4. State Management

4.1 Local State vs Global State

Local state (via `useState/useReducer`) lives inside one component and is relevant only to it and its direct children. **Global state** is data needed across many unrelated parts of the app — logged-in user info, theme, shopping cart, notifications. Choosing correctly matters: overusing global state for things that could stay local causes unnecessary re-renders and complexity.

Example 1 — appropriate local state

```
function Accordion() {
  const [isOpen, setIsOpen] = useState(false);
  // isOpen only matters to THIS accordion instance - keep it local
  return (
    <div>
      <button onClick={() => setIsOpen(!isOpen)}>Toggle</button>
      {isOpen && <p>Content here</p>}
    </div>
  );
}
```

Example 2 — appropriate global state

```
// Cart data is needed in Navbar (item count), CartPage, CheckoutPage,
// and ProductCard (add to cart button) - genuinely global
const CartContext = createContext();

function App() {
  const [cartItems, setCartItems] = useState([]);
  return (
    <CartContext.Provider value={{ cartItems, setCartItems }}>
      <Navbar /><ProductPage /><CheckoutPage />
    </CartContext.Provider>
  );
}
```

4.2 Context API

Context API is React's built-in solution for global state, avoiding third-party libraries for simpler apps. It works well for state that doesn't change extremely frequently (theme, auth, locale). For very frequent or complex state updates across a large app, dedicated libraries (Redux, Zustand) often perform better since Context re-renders ALL consuming components on any value change.

Example 1 — a complete auth context setup

```
const AuthContext = createContext();

function AuthProvider({ children }) {
  const [user, setUser] = useState(null);

  function login(userData) { setUser(userData); }
  function logout() { setUser(null); }

  return (
    <AuthContext.Provider value={{ user, login, logout }}>
      {children}
    </AuthContext.Provider>
  );
}

// wrap your whole app once:
//
```

Example 2 — consuming auth state anywhere

```
function Navbar() {
  const { user, logout } = useContext(AuthContext);
  return user
    ? <button onClick={logout}>Logout {user.name}</button>
    : <p>Not logged in</p>;
}
```

4.3 Redux / Zustand (Concept Overview)

For large apps with complex, frequently-updating shared state, companies often use dedicated libraries.

Redux — a single global store, updates via dispatched "actions" processed by pure "reducer" functions (same concept as `useReducer`, but app-wide). Requires more boilerplate but is very predictable and has excellent debugging tools.

Zustand — a lightweight, modern alternative with minimal boilerplate — just a hook-based store, no providers or reducers required.

Example 1 — Zustand store (much simpler syntax)

```
import { create } from "zustand";

const useCartStore = create((set) => ({
  items: [],
  addItem: (item) => set((state) => ({ items: [...state.items, item] })),
  clearCart: () => set({ items: [] })
}));

// used directly in ANY component, no Provider wrapping needed:
function Cart() {
  const { items, clearCart } = useCartStore();
  return <p>{items.length} items</p>;
}
```

Example 2 — Redux Toolkit (modern Redux, less boilerplate)

```
import { createSlice } from "@reduxjs/toolkit";

const cartSlice = createSlice({
  name: "cart",
  initialState: { items: [] },
  reducers: {
    addItem: (state, action) => { state.items.push(action.payload); }
  }
});

// dispatched with: dispatch(cartSlice.actions.addItem(newItem))
```

Which to learn first for jobs: Context API is built into React and enough for most small-to-medium apps and interviews. Redux (specifically Redux Toolkit) remains the most requested in job listings for larger companies; Zustand is increasingly popular for its simplicity in newer codebases.

5. Forms

5.1 Controlled Inputs

In React, forms are typically built with controlled inputs — each input's value is tied to state, and every keystroke updates that state via `onChange`. This gives you full control to validate, transform, or conditionally react to input as the user types.

Example 1 — a single controlled input

```
function NameForm() {
  const [name, setName] = useState("");

  function handleSubmit(e) {
    e.preventDefault(); // stop the page from reloading
    console.log("Submitted:", name);
  }

  return (
    <form onSubmit={handleSubmit}>
      <input value={name} onChange={e => setName(e.target.value)} />
      <button type="submit">Submit</button>
    </form>
  );
}
```

5.2 Handling Multiple Inputs

Instead of a separate `useState` for every single field (which gets repetitive), a common pattern uses ONE state object holding all fields, with a single generic `handleChange` function that updates whichever field triggered it — identified by its `name` attribute.

Example 1 — one handler for multiple fields

```
function SignupForm() {
  const [formData, setFormData] = useState({ name: "", email: "", password: "" });

  function handleChange(e) {
    const { name, value } = e.target;
    setFormData(prev => ({ ...prev, [name]: value })); // computed key updates the right field
  }

  return (
    <form>
      <input name="name" value={formData.name} onChange={handleChange} />
      <input name="email" value={formData.email} onChange={handleChange} />
      <input name="password" type="password" value={formData.password} onChange={handleChange} />
    </form>
  );
}
```

5.3 Form Validation Basics

Validation checks that input meets certain rules before submission — required fields, valid email format, minimum length, matching passwords. Simple validation can be done manually with conditions; for complex forms, libraries like React Hook Form or Formik handle validation, error state, and performance more efficiently.

Example 1 — manual validation

```
function LoginForm() {
  const [email, setEmail] = useState("");
  const [error, setError] = useState("");

  function handleSubmit(e) {
    e.preventDefault();
    if (!email.includes("@")) {
      setError("Please enter a valid email");
      return;
    }
    setError("");
    console.log("Submitting...");
  }

  return (
    <form onSubmit={handleSubmit}>
      <input value={email} onChange={e => setEmail(e.target.value)} />
      {error && <p style={{color:"red"}}>{error}</p>}
      <button type="submit">Login</button>
    </form>
  );
}
```

Example 2 — React Hook Form (industry standard library)

```
import { useForm } from "react-hook-form";

function LoginForm() {
  const { register, handleSubmit, formState: { errors } } = useForm();

  function onSubmit(data) { console.log(data); }

  return (
    <form onSubmit={handleSubmit(onSubmit)}>
      <input {...register("email", { required: true })} />
      {errors.email && <p>Email is required</p>}
      <button type="submit">Submit</button>
    </form>
  );
}

// less re-rendering, less boilerplate than manual useState per field
```

Why it matters for jobs: React Hook Form is extremely common in company codebases because it minimizes re-renders (unlike controlled inputs with `useState`, which re-render on every keystroke) and integrates well with validation schemas like Yup or Zod.

6. Side Effects & Data Fetching

6.1 useEffect Dependency Arrays (Deep Dive)

The dependency array tells React exactly WHEN to re-run an effect, by comparing each value in the array to its previous render's value (using reference equality, `===`). Getting this right is one of the most important React skills — wrong dependencies cause either infinite loops or stale, outdated data being used.

Example 1 — object/array dependencies are tricky

```
// PROBLEM: a new object is created every render, so this effect
// runs on EVERY render even though the "content" never changes
function Component() {
  const options = { limit: 10 }; // new reference every render!
  useEffect(() => {
    fetchData(options);
  }, [options]); // options is "different" every time, so this always re-runs
}

// FIX: move the object inside the effect, or memoize it, or use primitives
function ComponentFixed({ limit }) {
  useEffect(() => {
    fetchData({ limit });
  }, [limit]); // primitive value, compares correctly
}
```

Example 2 — multiple effects for separate concerns

```
function UserDashboard({ userId }) {
  // separate effects for separate, unrelated concerns - cleaner than one big effect
  useEffect(() => {
    fetchUser(userId);
  }, [userId]);

  useEffect(() => {
    document.title = `Dashboard - ${userId}`;
  }, [userId]);

  useEffect(() => {
    const interval = setInterval(refreshNotifications, 30000);
    return () => clearInterval(interval);
  }, []); // independent - runs once
}
```

6.2 Fetching APIs & Loading/Error States

A well-built data-fetching component tracks THREE states: loading (request in progress), error (something went wrong), and data (the successful result). Handling all three prevents confusing blank screens or crashes when things go wrong — this pattern is expected in virtually every production React app.

Example 1 — complete fetch pattern with all three states

```
function PostList() {
  const [posts, setPosts] = useState([]);
  const [loading, setLoading] = useState(true);
  const [error, setError] = useState(null);

  useEffect(() => {
    async function loadPosts() {
      try {
        setLoading(true);
        const res = await fetch("/api/posts");
        if (!res.ok) throw new Error("Failed to fetch");
        const data = await res.json();
        setPosts(data);
      } catch (err) {
        setError(err.message);
      } finally {
        setLoading(false);
      }
    }
    loadPosts();
  }, []);

  if (loading) return <p>Loading posts...</p>;
  if (error) return <p>Error: {error}</p>;
  return <ul>{posts.map(p => <li key={p.id}>{p.title}</li>)}</ul>;
}
```

Example 2 — avoiding race conditions with cleanup

```
function SearchResults({ query }) {
  const [results, setResults] = useState([]);

  useEffect(() => {
    let isCurrent = true; // flag to ignore outdated responses

    fetch(`/api/search?q=${query}`)
      .then(res => res.json())
      .then(data => {
        if (isCurrent) setResults(data); // only update if still relevant
      });

    return () => { isCurrent = false; }; // cancel relevance if query changes before response arrives
  }, [query]);

  // prevents an OLD, slow request from overwriting NEWER results
}
```

7. Performance Optimization

7.1 React.memo

`React.memo` wraps a component so React skips re-rendering it if its props haven't changed (shallow comparison). This is useful for components that render often but receive the same props most of the time — especially "expensive" ones (large lists, complex visuals).

Example 1 — basic `React.memo` usage

```
const ExpensiveList = React.memo(function ExpensiveList({ items }) {
  console.log("Rendering ExpensiveList");
  return <ul>{items.map(i => <li key={i.id}>{i.name}</li>)}</ul>;
});

function Parent() {
  const [count, setCount] = useState(0);
  const items = [{ id: 1, name: "Item" }]; // same reference issue as before!

  return (
    <>
      <button onClick={() => setCount(count+1)}>Clicked {count} times</button>
      <ExpensiveList items={items} />
    </>
  );
}
```

Example 2 — `memo` + `useMemo` working together

```
function Parent() {
  const [count, setCount] = useState(0);
  const [items] = useState([{ id: 1, name: "Item" }]);

  // memoize the array itself so it's the SAME reference across renders
  const stableItems = useMemo(() => items, [items]);

  return (
    <>
      <button onClick={() => setCount(count+1)}>+</button>
      <ExpensiveList items={stableItems} />
      // now ExpensiveList (wrapped in memo) truly skips re-rendering on count changes
    </>
  );
}
```

Gotcha: `React.memo` only helps if the props passed are stable (same reference) across renders. Passing a new inline object, array, or function every time defeats its purpose entirely — pair it with `useMemo`/`useCallback`.

7.2 Lazy Loading & Suspense

`React.lazy` lets you split your app's code into separate bundles, loading a component's code ONLY when it's actually needed — reducing initial load time. `<Suspense>` shows a fallback UI (like a spinner) while the lazy component's code is being downloaded.

Example 1 — lazy loading a heavy component

```
import { lazy, Suspense } from "react";

const HeavyChart = lazy(() => import("./HeavyChart"));

function Dashboard() {
  return (
    <Suspense fallback={<p>Loading chart...</p>}>
      <HeavyChart />
    </Suspense>
  );
}
// HeavyChart's code isn't downloaded until Dashboard actually renders it
```

Example 2 — lazy loading with routes (very common real usage)

```
const Home = lazy(() => import("./pages/Home"));
const Profile = lazy(() => import("./pages/Profile"));

function App() {
  return (
    <Suspense fallback={<Spinner />}>
      <Routes>
        <Route path="/" element={<Home />} />
        <Route path="/profile" element={<Profile />} />
      </Routes>
    </Suspense>
  );
}
// each page's JS is only downloaded when the user navigates to it
```

Why it matters: Companies care deeply about initial page load speed. Lazy loading routes/heavy components is standard practice in production React apps to keep the initial bundle small.

8. Routing

8.1 React Router Basics

React apps are Single Page Applications (SPAs) — normally there's only one HTML page, and JavaScript swaps content in and out. React Router is the standard library that simulates multi-page navigation (different URLs showing different components) without full page reloads.

Example 1 — basic router setup

```
import { BrowserRouter, Routes, Route } from "react-router-dom";

function App() {
  return (
    <BrowserRouter>
      <Routes>
        <Route path="/" element={<Home />} />
        <Route path="/about" element={<About />} />
        <Route path="/contact" element={<Contact />} />
      </Routes>
    </BrowserRouter>
  );
}
```

Example 2 — navigation with Link (not <a> tags)

```
import { Link } from "react-router-dom";

function Navbar() {
  return (
    <nav>
      <Link to="/">Home</Link>
      <Link to="/about">About</Link>
    </nav>
  );
}

// using .would cause a FULL PAGE RELOAD, defeating the purpose of an SPA
```

8.2 Dynamic Routes & URL Parameters

Dynamic routes let a single component handle many similar URLs, like `/products/1`, `/products/2`, etc. A colon in the path (`:id`) marks a placeholder; you read its actual value at runtime with the `useParams` hook.

Example 1 — defining and reading a dynamic route

```
import { useParams } from "react-router-dom";

// route definition:
// } />

function ProductDetail() {
  const { productId } = useParams(); // reads the actual value from the URL

  useEffect(() => {
    fetchProduct(productId);
  }, [productId]);

  return <p>Showing product #{productId}</p>;
}

// visiting /products/42 -> productId = "42"
```

Example 2 — query parameters (e.g. ?sort=price)

```
import { useSearchParams } from "react-router-dom";

function ProductList() {
  const [searchParams, setSearchParams] = useSearchParams();
  const sortBy = searchParams.get("sort") || "default";

  return (
    <button onClick={() => setSearchParams({ sort: "price" })}>
      Sort by price
    </button>
  );
}

// visiting /products?sort=price -> sortBy = "price"
```

8.3 Nested Routes

Nested routes let a parent layout component (with shared UI like a sidebar) render different children based on the URL, using an `<Outlet />` as the placeholder for whichever nested route matches.

Example 1 — nested route structure

```
import { Outlet } from "react-router-dom";

// route definitions:
// < > />
// } />
// } />
//

function DashboardLayout() {
  return (
    <div>
      <Sidebar />
      <main>
        <Outlet /> {/* renders Stats or Settings here, based on URL */}
      </main>
    </div>
  );
}

// /dashboard/stats renders DashboardLayout with Stats inside the Outlet
```

8.4 Programmatic Navigation

Sometimes you need to navigate in response to code (after a successful form submission, login, etc.) rather than a user clicking a link. `useNavigate` provides a function to do this from anywhere in a component.

Example 1 — redirecting after login

```
import { useNavigate } from "react-router-dom";

function LoginForm() {
  const navigate = useNavigate();

  async function handleLogin() {
    const success = await loginUser();
    if (success) {
      navigate("/dashboard"); // redirect programmatically
    }
  }
}
```

Example 2 — navigate with replace (no back-button history)

```
navigate("/dashboard", { replace: true });
// replaces current history entry - user can't hit "back" to return to login page
```

9. Advanced Patterns

9.1 Higher-Order Components (HOC)

A Higher-Order Component is a function that takes a component and returns a NEW, enhanced component — adding extra props or behavior without modifying the original. This was the standard way to share logic before hooks existed; you'll still see it in older/legacy codebases.

Example 1 — a basic HOC adding a loading state

```
function withLoading(WrappedComponent) {
  return function(props) {
    if (props.isLoading) return <p>Loading...</p>;
    return <WrappedComponent {...props} />;
  };
}

function UserList({ users }) {
  return <ul>{users.map(u => <li key={u.id}>{u.name}</li>)}</ul>;
}

const UserListWithLoading = withLoading(UserList);
// usage:
```

Example 2 — HOC for auth protection (common real use)

```
function withAuth(WrappedComponent) {
  return function(props) {
    const { user } = useContext(AuthContext);
    if (!user) return <Navigate to="/login" />;
    return <WrappedComponent {...props} />;
  };
}

const ProtectedDashboard = withAuth(Dashboard);
// note: a custom hook (useAuth) is now more common for this exact use case
```

HOCs vs Custom Hooks today: Custom hooks have replaced most HOC use cases in modern React because they avoid the extra wrapper component layer and "wrapper hell" from stacking multiple HOCs. Still worth knowing since many companies have legacy code using HOCs.

9.2 Render Props

The render props pattern shares logic by passing a FUNCTION as a prop, which the component calls to determine what to render — giving the consumer full control over the UI while the component handles the logic.

Example 1 — a mouse-tracker with render props

```
function MouseTracker({ render }) {
  const [position, setPosition] = useState({ x: 0, y: 0 });

  function handleMouseMove(e) {
    setPosition({ x: e.clientX, y: e.clientY });
  }

  return (
    <div onMouseMove={handleMouseMove}>
      {render(position)} {/* calls the function passed in, with current data */}
    </div>
  );
}

// usage - consumer decides exactly how to display the position:
<MouseTracker render={(pos) => <p>X: {pos.x}, Y: {pos.y}</p> } />
```

Render props vs Custom Hooks: A custom hook ([useMousePosition\(\)](#)) achieves the same sharing of logic more simply in modern React, without needing a wrapper component. Render props are still seen in some libraries' APIs though.

9.3 Compound Components

Compound components let several components work together implicitly, sharing state via context internally, while giving a clean, declarative API to whoever uses them (similar to how native `<select>` and `<option>` work together).

Example 1 — a compound Tabs component

```
const TabsContext = createContext();

function Tabs({ children, defaultTab }) {
  const [activeTab, setActiveTab] = useState(defaultTab);
  return (
    <TabsContext.Provider value={{ activeTab, setActiveTab }}>
      {children}
    </TabsContext.Provider>
  );
}

Tabs.Tab = function({ id, children }) {
  const { activeTab, setActiveTab } = useContext(TabsContext);
  return (
    <button onClick={() => setActiveTab(id)} style={{fontWeight: activeTab===id ? "bold" : "normal"}}>
      {children}
    </button>
  );
};

Tabs.Panel = function({ id, children }) {
  const { activeTab } = useContext(TabsContext);
  return activeTab === id ? <div>{children}</div> : null;
};
```

Example 2 — using the compound component (clean, readable API)

```
<Tabs defaultTab="home">
  <Tabs.Tab id="home">Home</Tabs.Tab>
  <Tabs.Tab id="profile">Profile</Tabs.Tab>

  <Tabs.Panel id="home">Home content</Tabs.Panel>
  <Tabs.Panel id="profile">Profile content</Tabs.Panel>
</Tabs>

// state is shared internally via context - the consumer doesn't manage it at all
```


10. Ecosystem Essentials

10.1 Error Boundaries

An [Error Boundary](#) is a component that catches JavaScript errors anywhere in its child component tree, logs them, and displays a fallback UI instead of crashing the whole app. Error boundaries must currently be written as CLASS components — there's no hook equivalent yet — but you typically only write one and reuse it everywhere.

Example 1 — a reusable Error Boundary

```
class ErrorBoundary extends React.Component {
  constructor(props) {
    super(props);
    this.state = { hasError: false };
  }

  static getDerivedStateFromError() {
    return { hasError: true };
  }

  componentDidCatch(error, info) {
    console.log("Error caught:", error, info);
  }

  render() {
    if (this.state.hasError) {
      return <h2>Something went wrong.</h2>;
    }
    return this.props.children;
  }
}
```

Example 2 — using it to isolate failures

```
function App() {
  return (
    <ErrorBoundary>
      <Header />
      <ErrorBoundary>
        <RiskyWidget /> {/* if this crashes, only it fails, not the whole app */}
      </ErrorBoundary>
      <Footer />
    </ErrorBoundary>
  );
}
```

10.2 Keys & Reconciliation Deep Dive

React's reconciliation algorithm decides what to update in the real DOM by comparing the new virtual tree with the old one. For lists, [key](#) is the hint React uses to match old items to new ones — WITHOUT a stable key, React may unnecessarily destroy and recreate DOM nodes (losing input focus, animation state, etc.) instead of just reordering them.

Example 1 — the visible cost of wrong keys

```
// each TodoItem has its own local "isEditing" state
function TodoItem({ text }) {
  const [isEditing, setIsEditing] = useState(false);
  return <li>{text} {isEditing && <input />}</li>;
}

// if todos are reordered and keys are array indices, React may
// incorrectly reuse a DOM node with a DIFFERENT item's local state attached
{todos.map((todo, i) => <TodoItem key={i} text={todo.text} />)} // risky

{todos.map((todo) => <TodoItem key={todo.id} text={todo.text} />)} // correct
```

10.3 Component Testing Basics

Companies expect basic familiarity with testing React components. The standard tools are **Jest** (test runner/assertions) and **React Testing Library** (renders components and simulates user interaction, encouraging tests that check behavior rather than implementation details).

Example 1 — a simple component test

```
import { render, screen, fireEvent } from "@testing-library/react";

function Counter() {
  const [count, setCount] = useState(0);
  return <button onClick={() => setCount(count+1)}>Count: {count}</button>;
}

test("increments count on click", () => {
  render(<Counter />);
  const button = screen.getByText("Count: 0");
  fireEvent.click(button);
  expect(screen.getByText("Count: 1")).toBeInTheDocument();
});

// tests behavior (what the user sees/does) not internal implementation
```

End of guide | You now have theory + examples for every major concept companies expect in React interviews and production codebases. Practice by rebuilding small features (a todo app, a search-as-you-type box, a paginated product list) using these patterns together.